FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

# CS0065
# (INTELLIGENT SYSTEMS)

## TECHNICAL ASSESSMENT

# 4

## IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

| | |
|---|---|
| **Student Name** | Jed L. Hizon<br>Rembrandt V. Coritana |
| **Section:** | AN41 |
| **Professor:** | Ms. Crisola G. Tan |
| **Date:** | Oct. 3, 2026 |

## I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

**1. Rule-Based Reasoning (RBR)**

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

**2. Case-Based Reasoning (CBR)**

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

## II. TECHNICAL ASSESSMENT

### IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

***Objective***

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

**Materials Needed**

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

**General Instructions**

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    *(Example: TA2_Patilano.pdf)*
11. Submit the **python file** and **PDF** file.

**LABORATORY TASKS**

**PART 1 — Knowledge Representation (KR) Setup**

**Task 1A — Create a Knowledge Base**

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

**Example structure to follow:**

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

**PART 2 — Implement Rule-Based Reasoning (RBR)**

**Task 2A — Create at least 3 IF–THEN rules**

Write Python functions representing rules.

**Example:**

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

**Task 2B — Implement an Inference Engine**

Create a loop that:

1. Reads the knowledge base

2. Checks each rule
3. Returns all actions inferred

**Task 2C — Display Output**

**Example [Program should print:]**

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

**PART 3 — Implement Case-Based Reasoning (CBR)**

**Task 3A — Build a Case Base**

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

**Example:**

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

**Task 3C — Reuse Step**

Return the solution of the most similar case.

**Task 3D — Revise Step**

Allow student to adjust solution manually or return unchanged.

**Task 3E — Retain Step**

Add the new case to the case base.

**PART 4 — Comparative Reflection**

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
    - In our opinion, RBR was easier to implement. Each rule is a simple IF–THEN function, and the inference engine is just a loop that checks every rule and collects the actions that fire. While CBR needed more parts: a case base, a similarity function to compare problems, and the full 4R cycle (Retrieve, Reuse, Revise, Retain).

2. Which is more flexible in solving unstructured problems?
    - CBR is more flexible. It does not need a rule for every situation. When a new problem has no exact match, it retrieves the closest past case and reuses its solution.

3. Which is better for situations with continuously changing data?
    - RBR is better when the data changes constantly, such as CPU temperature or battery level. The rules are re-evaluated against the current facts each time the engine runs, so the output always reflects the latest values.
4. Examples of real-world applications for both.
    - RBR
        - Medical diagnosis support systems (for example, MYCIN)
        - Loan or tax eligibility checkers
    - CBR
        - Legal systems that look up similar court cases
        - Recipe and product recommenders
        - Equipment fault diagnosis that matches new faults to past repairs

**Your Code - Laboratory Tasks** ⇩

```
# Part A: Knowledge Representation + Rule-Based Reasoning (RBR)
# Use case: Laptop troubleshooting assistant

facts = {
    "power": {
        "powers_on": True,
        "battery_level": 12,        # percent
        "is_plugged_in": False,
    },
    "thermal": {
        "temperature_c": 92,        # CPU temperature
        "fan_noise": "loud",        # "normal" or "loud"
    },
    "display": {
        "screen_on": True,
        "screen_flickering": False,
        "storage_free_gb": 3,       # included for the storage rule
    },
}

def rule_no_power(facts):
                    if    not    facts["power"]["powers_on"]    and    not
facts["power"]["is_plugged_in"]:
        return "Plug in the charger and check the power outlet"
```

```
def rule_low_battery(facts):
              if    facts["power"]["battery_level"]    <    20    and    not
facts["power"]["is_plugged_in"]:
        return "Battery is low - plug in the charger"

def rule_overheating(facts):
    if facts["thermal"]["temperature_c"] > 85:
          return "Close heavy programs and place the laptop on a cooling
pad"

def rule_loud_fan(facts):
    if facts["thermal"]["fan_noise"] == "loud":
        return "Clean the fan and air vents from dust"

def rule_flicker(facts):
    if facts["display"]["screen_flickering"]:
        return "Update the graphics/display driver"

def rule_low_storage(facts):
    if facts["display"]["storage_free_gb"] < 10:
        return "Delete unused files or run Disk Cleanup"

# The rule base (the list the inference engine will go through)
rules = [rule_no_power, rule_low_battery, rule_overheating,
         rule_loud_fan, rule_flicker, rule_low_storage]


# Task 2B: Inference engine
def inference_engine(facts, rules):
      # Read the knowledge base, check each rule, return all inferred
actions.
    actions = []                        # working memory of conclusions
    for rule in rules:                  # check every rule
        result = rule(facts)            # evaluate the IF part
        if result is not None:          # rule fired -> keep the THEN part
            actions.append(result)
```

```
    return actions


# Task 2C: Display output
if __name__ == "__main__":
    print("Knowledge Base (Facts):")
    for category, items in facts.items():
        print(f"  [{category}]")
        for name, value in items.items():
            print(f"    {name}: {value}")

    print("\nRule-Based Reasoning Actions:")
    inferred = inference_engine(facts, rules)
    if inferred:
        for action in inferred:
            print(f"- {action}")
    else:
        print("- No rule matched. No action needed.")
```

```
Knowledge Base (Facts):
  [power]
    powers_on: True
    battery_level: 12
    is_plugged_in: False
  [thermal]
    temperature_c: 92
    fan_noise: loud
  [display]
    screen_on: True
    screen_flickering: False
    storage_free_gb: 3

Rule-Based Reasoning Actions:
- Battery is low - plug in the charger
- Close heavy programs and place the laptop on a cooling pad
- Clean the fan and air vents from dust
- Delete unused files or run Disk Cleanup
```

```
# Part B: Case-Based Reasoning (CBR)
```

```
# Use case: Laptop troubleshooting assistant (same domain as the RBR program)

case_base = [
    {"problem": {
        "wont_turn_on": True,
        "overheating": False,
        "loud_fan": False,
        "slow": False,
        "screen_flicker": False
        },
     "solution": "Check the power cable and charger."
     },
    {"problem": {
        "wont_turn_on": False,
        "overheating": True,
        "loud_fan": True,
        "slow": True,
        "screen_flicker": False
        },
     "solution": "Clean the fan and vents, then use a cooling pad."
     },
    {"problem": {
        "wont_turn_on": False,
        "overheating": False,
        "loud_fan": False,
        "slow": True,
        "screen_flicker": False
        },
     "solution": "Free up disk space and remove startup programs."
     },
    {"problem": {
        "wont_turn_on": False,
        "overheating": False,
        "loud_fan": False,
        "slow": False,
        "screen_flicker": True
```

```
        },
     "solution": "Update the display driver."
     },
]

FEATURES = [
    "wont_turn_on",
    "overheating",
    "loud_fan",
    "slow",
    "screen_flicker"
    ]


# Retrieve - similarity matching
def similarity(new_problem, old_problem):
    # Fraction of features that have the same value (0.0 to 1.0).
    matches = sum(1 for f in FEATURES if new_problem[f] == old_problem[f])
    return matches / len(FEATURES)

def retrieve(new_problem, case_base):
    # Return the most similar case and its score.
    best_case, best_score = None, -1
    for case in case_base:
        score = similarity(new_problem, case["problem"])
        if score > best_score:
            best_case, best_score = case, score
    return best_case, best_score


# Reuse - use the solution of the most similar case
def reuse(best_case):
    return best_case["solution"]


# Revise - let the student adjust the solution or keep it
def revise(solution):
```

```
    print(f"\nSuggested solution: {solution}")
     answer = input("Edit the solution? (type new solution or press Enter
to keep): ").strip()
    return answer if answer else solution


# Retain - store the new case in the case base
def retain(new_problem, final_solution, case_base):
    case_base.append({"problem": new_problem, "solution": final_solution})
    print(f"Case retained. Case base now has {len(case_base)} cases.")


def get_yes_no(prompt):
    # Keep asking until the user enters a valid yes/no answer
    while True:
        answer = input(prompt).strip().lower()
        if answer in ["y", "yes"]:
            return True
        elif answer in ["n", "no"]:
            return False
        else:
            print("Invalid input! Please enter 'y' or 'n'.")


def ask_problem():
    # Input: ask the user yes/no questions to build the new problem.
    print("Describe your laptop problem (y/n):")
    problem = {}
    for f in FEATURES:
        problem[f] = get_yes_no(f"  {f.replace('_', ' ')}? ")
    return problem

def main():
    print("=== Case-Based Reasoning: Laptop Troubleshooter ===\n")
    new_problem = ask_problem()

    best_case, score = retrieve(new_problem, case_base)
```

```
    print(f"\nMost similar case found (similarity: {score:.0%}):")
    print(f"  Problem : {best_case['problem']}")

    solution = reuse(best_case)
    final_solution = revise(solution)
    retain(new_problem, final_solution, case_base)

    print(f"\nFinal solution: {final_solution}")

if __name__ == "__main__":
    main()
```

```
=== Case-Based Reasoning: Laptop Troubleshooter ===

Describe your laptop problem (y/n):
  wont turn on? a
Invalid input! Please enter 'y' or 'n'.
  wont turn on? y
  overheating? y
  loud fan? n
  slow? y
  screen flicker? n

Most similar case found (similarity: 60%):
  Problem : {'wont_turn_on': True, 'overheating': False, 'loud_fan': False, 'slow': False, 'screen_flicker': False}

Suggested solution: Check the power cable and charger.
Edit the solution? (type new solution or press Enter to keep):
Case retained. Case base now has 5 cases.

Final solution: Check the power cable and charger.
```